Stigmergic Project Delivery: Dynamic Specification-Driven Planning and Trace-Based Forecasting

Matthew Fyfe • October 2026

Abstract

Project planning and delivery rely on processes designed manually, in isolation from product specification and estimation and decoupled from execution, with forecasts ungrounded in resource-level execution data. Each of these areas is well studied, but to our knowledge no single model derives process directly from a specification while also forecasting outcomes and allocating work among agents. We propose Stigmergic Project Delivery (SPD), a formal model in which a specification generates process structure that remains dynamic during execution. In the model, execution leaves in that process a trace that drives both forecasting and self-selected allocation among agents, without central control. We explain how SPD instantiates Heylighen's components of stigmergy, with coordination emerging from them, and compare it against product-based workflow design, Activity-on-Node scheduling and predictive process monitoring to identify what each leaves unresolved. By deriving both risk-aware forecasts and task allocation directly from the specification and trace, SPD is intended to improve the predictability of delivery and reduce the forecasting and coordination overhead carried by management layers.

1. Introduction

Project planning methods typically separate three functions that ought to inform one another. They derive process structure by hand rather than from a specification of the product; they forecast performance from assumed or generic distributions, or from project-level history, rather than from a recorded history of the agents, tools, and materials involved; and they coordinate who does the work through central assignment, rather than letting it fall out of the work's own structure.

This paper takes the position that projects are drawn from an operational domain that exists before and beyond any of them. *Products* are delivered by operations afforded by the *materials* worked and bounded by the capabilities of the *tools* and *agents* that work them. Products and operations alike belong to the domain, not to any one *project*. A project specifies a particular product and draws on the operations its delivery requires.

This position is what lets the three functions inform one another. Process can be derived rather than designed, because the operations a product needs already exist and only have to be selected. History can accumulate, because the same operations recur across projects even when the projects themselves are unique. Allocation can follow from the trace, because an agent's experience belongs to the domain, recorded against operations that outlast any one project.

This paper's contribution is Stigmergic Project Delivery (SPD), a single model built on that position. It represents the product side, the space of possible products and their variation, as the *master specification*, and the operational side, the operations available to deliver them, as the *master process*, with a *mapping* between the two, so that a selection within the product yields the process that delivers it. Process is derived from a specification, and trace-based *forecasting* and *self-selected* task allocation both draw on the *trace* of previous executions.

Each project begins with a provided instantiation of the specification, a subset of the master with its *descriptors* bound, from which the corresponding subset of the master process is derived. The order of steps follows from the products routed between them rather than being specified by hand, as Activity-on-Node scheduling requires. The specification and its derived process remain dynamic and extensible during execution. From the trace of earlier projects, each derived step carries a forecast of its delivery metrics, material usage, time, and whatever else the domain records; the same trace fixes which agents are *eligible* for the job it belongs to.

Eligible agents self-select to take up each job's *contract*, and as they work, data on those metrics accrues back into the trace, manually or via telemetry. The trace persists across projects, closing the loop: each project forecasts and allocates from the history before it, and leaves its own for the next.

Section 2 positions SPD against prior work along the three legs it combines. Section 3 gives the formal model: the specification, the mapping, the trace, the eligibility and self-selection they drive, and the contract it produces. Section 4 works two concrete examples by hand. Section 5 takes each source of data in turn, reporting what has been tested, what it can still test, and what it cannot. Section 6 discusses the model's limitations. Section 7 concludes.

2. Related work

We position SPD against prior work on two fronts. The first is the three legs it combines, deriving process from specification, forecasting from a trace, and coordinating allocation through that same trace, the last an instance of Heylighen's stigmergy. The second is the two formalisms it extends, the activity-on-node process graph and the classical resource model.

Process derivation from specification. The idea that process structure should follow from product structure, rather than being authored independently, originates with van der Aalst [1] and was developed into product-based workflow design (PBWD) by Reijers, Limam, and van der Aalst [2], who derive a workflow from a product specification and design criteria using formal methods. PBWD's product data model fixes a single cost, time, and probability per production rule at design time, and cannot represent repeated components. SPD addresses both issues, building on the and/or feature tree of Kang et al. [3] and its later cardinality-based extensions [4]. The split between master and project echoes software product line engineering, which separates domain engineering, building the reusable space of features and assets, from application engineering, deriving a single product from it [5]. Product lines formalise that space and the derivation of a product from it, but they do not record how each derived product's work was carried out, by which agents, tools and materials, nor forecast or allocate from that record, which is where SPD extends them.

Rolling specification and continuous derivation. A further limitation of PBWD is temporal rather than structural: it derives a workflow once, from a fixed specification, and does not accommodate a specification that changes after execution begins. Case handling [6] takes the opposite stance, deriving what can be done from case data as it arrives rather than from a pre-specified control flow, but PBWD's own authors found the two could not be bridged: current case handling could not adequately support product-based design [7]. SPD does both at once: its derivation is continuous, so an extension to the specification during execution extends the derived process with it. Section 4 documents this in full.

Network formalisms. SPD's process graph is best understood as a modified Activity-on-Node graph [8], where edges are promoted to first-class objects representing the products passed between the activities. That typing gives each step a contract, the inputs it consumes and the outputs it must produce, which is what an agent takes on and is held to, where a precedence link expresses only ordering and so offers nothing to commit to. The typed product is the producer/consumer dependency of coordination theory, treated below. It also carries the product's own properties, such as the time a crop needs to mature before harvest, which a downstream step reads from the product it receives rather than from a lag attached to the link.

The graph captures each agent's internal process, not only the hand-offs between agents. Within a job, a step's products often pass to the next step rather than crossing to another agent, and the typed edge records which, where a precedence link does not.

Resources and agency. No existing academic resource formalism captures the three-way split SPD needs, self-directing agentic resources, tools granted for use, and materials embodied in the product: the Resource-Task Network [9, 10] models materials, equipment, and labour within a single uniform abstraction, and multi-

agent systems research [11] separates autonomous agents from passive artifacts but not materials. The closest match is an industry taxonomy rather than an academic one, the six construction-resource subtypes Industry Foundation Classes [12] standardise: IfcLaborResource, IfcCrewResource, and IfcSubContractResource fold into agentic; IfcConstructionEquipmentResource is tools; and IfcConstructionMaterialResource and IfcConstructionProductResource are materials. In all of them, IFC included, labour is a resource the model allocates; SPD's extension is to treat it as an agent rather than an allocatable resource.

Trace-based forecasting. Predictive process monitoring, surveyed by Márquez-Chamorro, Resinas, and Ruiz-Cortés [13], forecasts properties of an ongoing process instance (remaining time, cost, or outcome) from historical event logs, and has been shown to benefit from resource- and workload-specific features. This establishes that forecasting from resource-attributed history is well-developed in its own right, but it forecasts within a process it assumes already exists, rather than deriving that process. Within project management specifically, schedule risk analysis [14-16] measures activity and resource sensitivity via Monte Carlo simulation over the network, typically using distributions fitted to subjective three-point estimates [17] rather than measured performance of specific agents, tools, or materials. Vanhoucke and colleagues have moved this line from expert-estimated inputs towards measured history: machine learning trained on past projects [18] and nearest-neighbour matching against comparable ones [19] both forecast from recorded outcomes. Matching against comparable projects is reference-class forecasting [20], which predicts from the outcomes of past cases similar to the one at hand rather than from expert judgment; Batselier and Vanhoucke evaluate it for project management and integrate it with earned value management [21, 22], drawing on the empirical project database they document [23]. These methods learn at the level of the project or activity, the level their data sits at; but such data is itself hard to come by, the field often leaning on artificial data [24], and history at the granularity of individual agents, tools, and materials is scarcer still. SPD closes this gap by forming the reference class at any level the work is decomposed to, from the job down to the individual action, not only the whole project: the finer units recur across otherwise-unique projects, so their history accrues as a by-product of execution where sparse, rarely-repeated project history does not. Recording at a finer level moves uncertainty toward the irreducible aleatory variation that rules out deterministic forecasting [25], though each finer context also holds fewer matching records, so the gain is a direction of travel rather than a floor reached.

Coordination mechanism. Coordination theory [26] frames coordination as the management of dependencies between activities. The products passed between SPD's steps are its producer/consumer dependency, which SPD derives from the product specification and executes, where the theory classifies the dependencies of a process already given.

Stigmergy, coordination through traces left in a shared medium rather than direct instruction, was first described by Grassé [27] and given formal treatment by Heylighen [28, 29], who defines it through five components: action, agent, medium, trace, and coordination. SPD provides the first four: an action is a step's execution; agents are the executors; the medium is the process graph; and the trace is the record that execution leaves. Stigmergic coordination emerges from these as the resulting self-selection: agents are filtered to an eligible set by matching recorded experience against each job's requirements, and the forecast attached to the step, drawn from its trace, is the stimulus an eligible agent then chooses on, rather than being centrally assigned. One of Heylighen's own illustrations of stigmergic coordination among humans, programmers self-selecting from a shared, largely independent backlog, demonstrates the same underlying principle in its simplest form; SPD extends it to a structurally derived graph of dependent steps, each bound by a contract between agent and product, rather than an unstructured or independent task.

The same coordination mechanism is attested in construction, a domain of structured projects with interdependent skillsets rather than isolated tasks. Christensen [30] observes builders coordinating design and site work by acting on the physical evidence of work already accomplished by others, rather than through direct instruction. That is the mechanism in its reactive form, the stimulus being the realised state of completed work.

SPD proposes a structure in which the same principle operates anticipatorily, each step carrying a forecast drawn from the trace, so an agent selects on a prediction of the step's delivery rather than on the residue of what is already done.

Where the same practice has been measured, in software, its informal human form shows clear shortcomings. Masood, Hoda, and Blincoe [31, 32] find that self-assignment in agile teams is driven by informal, individually varying preferences, task-based, developer-based, and opinion-based; where an agent's own capability is considered at all, it enters as a self-assessed and inconsistently applied factor rather than a structured record of demonstrated performance on comparable prior work. Estimation is no more reliable: as Conoscenti et al. [33] find, developer inexperience is among the commonly reported causes of inaccurate estimates, and structured reflection on one's own completed tasks improves neither estimation accuracy nor the realism of uncertainty assessments, while the same feedback about other professionals' estimates does make uncertainty assessments more realistic [34]. SPD addresses these shortcomings for the structured-project domain the construction evidence points to: it replaces self-assessed capability with the trace's record of demonstrated performance, and subjective estimates with forecasts measured from it.

Experience-based ranking within eligibility. Eligibility admits the agents holding a job's skillset with enough experience at each of its steps. Among them, experience with the particular attributes of the job's steps sets the order of claim. Two agents may share the same skillset yet differ in their recorded history against a given descriptor, one having completed several prior instances with a specific specialism, the other none. The trace is queried for each eligible agent's history of similar prior steps, and the most and best-distributed record is favoured in self-selection, rather than the job being centrally assigned. This resembles history-based task assignment in automated bug triage [35], where candidates within a pre-qualified pool are ranked not by declared expertise but by their record of resolving previous instances with matching attributes.

3. The model

The model formalises three things, the derivation of process from a specification, trace-based forecasting, and trace-driven preferential self-selection among eligible agents. SPD concerns the execution of projects, which run on a given master specification, process, and mapping.

3.1 Definitions

Product. What a unit of work delivers, at different levels of granularity. The same product can be output by a step, by a job, and by the whole process. Many serve only the next step within the same job; others cross a job boundary to serve a step in another job; and one will ultimately leave its step, its job, and the whole process, fulfilling the obligation of the specification.

Master specification. A rooted tree that breaks the product down into all its possible elements. The tree is hierarchical: the first level holds the high-level components, those delivered by whole systems, such as a building's foundation or its structural walls; the next level holds the systems that can deliver each component, a foundation, for instance, by a concrete slab or by blockwork; lower levels break the systems down as far as needed. Write it $T = (N, A)$: N the product elements and A the parent-child relations. A node may carry descriptors, as defined below.

$$T = (N,\ A) \tag{1}$$

Master process. A directed graph that breaks delivery down into all its possible steps. The nodes are the steps; the arcs each carry a product passed between two steps, thereby determining their precedence in an acyclic graph. Write it $G = (N_s, A_{sp})$: N_s the steps, A_{sp} the step products, a single step written s, and an arc $e = (u, v)$ carrying a product from step u to step v.

$$G = (N_s,\ A_{sp}) \qquad e = (u, v) \in A_{sp} \tag{2}$$

Job graph. A graph grouping the steps by job, each job defined by one skillset. It may cycle where products pass back and forth between jobs. Write it $G_j = (N_j, A_{jp})$: N_j the jobs and A_{jp} the job products. Steps are assigned to jobs by a grouping $J : N_s \to N_j$, and a job-arc joins two jobs wherever a product passes between their steps.

$$G_j = (N_j,\ A_{jp}) \qquad J : N_s \to N_j$$

Every step belongs to exactly one job. (3)

$$A_{jp} = \{\, (J(u),\ J(v)) : (u, v) \in A_{sp},\ J(u) \neq J(v) \,\} \tag{4}$$

Mapping. A deterministic association of each node of the master specification with the steps in the master process it necessitates: a given product node always necessitates the same steps. A step may be necessitated by more than one node, so the steps of different nodes may overlap. Write it as a map $\mu : N \to 2^{N_s}$, from spec nodes to the sets of steps they necessitate.

$$\mu : N \to 2^{N_s}$$

Each node of the specification necessitates a set of steps. (5)

Descriptor. An attribute attached to a node of the master specification, numerical or categorical. A node carries descriptors $D = (d_1, \ldots, d_k)$; on an instance each takes a value, and the metrics at the steps the node necessitates scale with those values, as the forecast below makes precise.

Derivation. A project's nodes are a subset $P \subseteq N$ of the master specification, closed upward so selecting a node selects its ancestors up to the root, with their descriptors bound to values. The root is therefore always present, and it necessitates the steps the product needs as a whole, estimation being a common example. Each selected node necessitates its steps through the mapping, and pooled together these are the project's steps N_P, with the project's process G_P the master process induced on them. The specification can be extended during execution, drawing further subsets the same way. Extension can come from outside, a client's change, say, or from execution itself, where a step's finding selects further nodes, as an assessment selects the organ it localises. Write the nodes a step's execution selects as $\sigma(s) \subseteq N$. Executing s then extends the selection to $P \cup \sigma(s)$, closed upward, and the steps those nodes necessitate join the project's process. Extension only adds nodes, so a larger selection$P \subseteq P'$ gives $N_P \subseteq N_{P'}$ and makes G_P a subgraph of $G_{P'}$. Steps already executed, and the records they left, therefore stay valid, unless a newly necessitated step feeds one of them, a limitation Section 6 takes up.

$$G_P = G[\, N_P \,] \qquad N_P = \bigcup_{x \in P} \mu(x)$$

A project's process is the master process restricted to its steps. *A project's steps are those its selected nodes necessitate.* (6)

Agent. A resource that performs steps, as against the tools and materials a step also draws on; it holds a set of skillsets $Q(a)$, its qualifications.

Tool. A resource a step draws on without embodying it in the product, such as equipment, software, a vehicle, or a consumable like sandpaper or a face mask, whether used up in a single step or worn down over many, written g.

Material. A resource a step embodies in the product, written w.

Trace. An accumulating set of execution records. Each ties the value y of one metric m recorded at a step s at time t, under the descriptor values D that held, to the agent a that executed it, the tools g and materials w it drew on, and the project instance i it arose from. An execution leaves a record for each metric it captures, so steps can record different metrics, each when and however it is captured. Beyond the project, step, time, metric and value, any field may go unrecorded, written $\perp$, as distinct from an empty set of tools or materials, which records that none were used. A record then serves only the forecasts and exposure its recorded fields can answer, so a partial trace is useful in proportion to what it records.

$$\tau = \left\{ (i,\ s,\ t,\ D,\ a,\ g,\ w,\ m,\ y) \right\} \tag{7}$$

Exposure. The recorded exposure to a step of anything the trace records, alone or in combination, written $n_c(s)$ for such a context c, as a count of executions or a total of some recorded quantity, say. Its reading follows the context, wear for a tool, for instance, experience for an agent, written $n_a(s)$, or how often the step has run at night.

Eligibility. An agent is eligible for a job it is qualified and has sufficient experience for, according to a given per-step floor λ_s at each of the job's steps.

$$\mathrm{Elig}(j) = \left\{ a : j \in Q(a),\ n_a(s) \geq \lambda_s \text{ for all } s \text{ with } J(s) = j \right\} \tag{8}$$

Forecast. A predicted value for one of a step's metrics, based on the trace. A metric's value is distributed as $y_m(s) \sim \mathcal{F}(\theta(D), s, c)$, conditioned on a chosen context c. The descriptors D a step sees are those bound on the nodes that necessitate it and on their ancestors. Here c selects the records the fit draws on, θ scales within them by the descriptors, and a neighbourhood fixes which descriptor values count as near, $D' \approx D$. What c holds follows the use. The estimate offered against a step takes c empty, pooling over who will do the work, so a client is not charged for drawing the slowest worker. Planning can condition more finely, with c naming the worker, the tool, the time of day, and so on. The forecast reads off the fit over the records that match the step and metric, lie near its descriptor values, and agree with c, at a tolerated risk r, its r-percentile Q_r. A lower tolerance for overrun takes a higher percentile, a more conservative value.

$$y_m(s) \ \sim\ \mathcal{F}\big(\theta(D),\ s,\ c\big) \tag{9}$$

A metric's value at a step follows a distribution scaled by its descriptors and conditioned on the chosen context.

$$\mathrm{forecast}(s, m, D, r, c) = Q_r\big(\ \mathrm{fit}\left\{ y : (i', s, t', D', a', g', w', m, y) \in \tau,\ D' \approx D,\ \mathrm{match}(c) \right\}\big) \tag{10}$$

The forecast for a metric is the r-percentile of a fit over its matching records near the step's descriptor values.

Selection. Among the agents eligible for a job, each self-selects rather than being assigned, weighing the job's forecasts at its own tolerated risk. Experience across the job's steps sets their priority, favouring the most and best-distributed record, so preference emerges from the trace rather than from central assignment.

Contract. The unit offered for self-selection, a job within the derived subgraph: the inputs $\mathrm{in}(j)$ it draws from outside, including any whose producing step is not selected, the outputs $\mathrm{out}(j)$ it supplies, and the forecasts $\varphi(j)$, for each metric the r-percentile of the distribution obtained by combining the job's steps under independence, not a sum of per-step percentiles. A contract $C(j)$ is offered to every agent in $\mathrm{Elig}(j)$ and becomes binding on whichever self-selects it.

$$C(j) = \big(\ \mathrm{in}(j),\ \mathrm{out}(j),\ \varphi(j)\ \big) \tag{11}$$

Taken together, a project is each of its derived steps with that step's forecasts across its metrics, and the contract of the job it belongs to with the agents eligible for that job, offered for self-selection:

$$\mathrm{Project}(P) = \left\{ \left(s,\ \mathrm{forecast}(s, \cdot, D, r, \varnothing),\ C(J(s)),\ \mathrm{Elig}(J(s))\right) : s \in N_P \right\} \tag{12}$$

Throughout, N, N_s and the agents are taken finite, and the trace τ is finite at any point. The master process G is acyclic, while the job graph G_j may cycle.

3.2 Illustration

Figure 1 draws the model in miniature, a master specification beside its master process. One project selects the root S with A and B but not C, and the mapping picks out the steps those nodes necessitate, the project's process G_P. What the project uses is drawn solid, and the rest of the master faded.

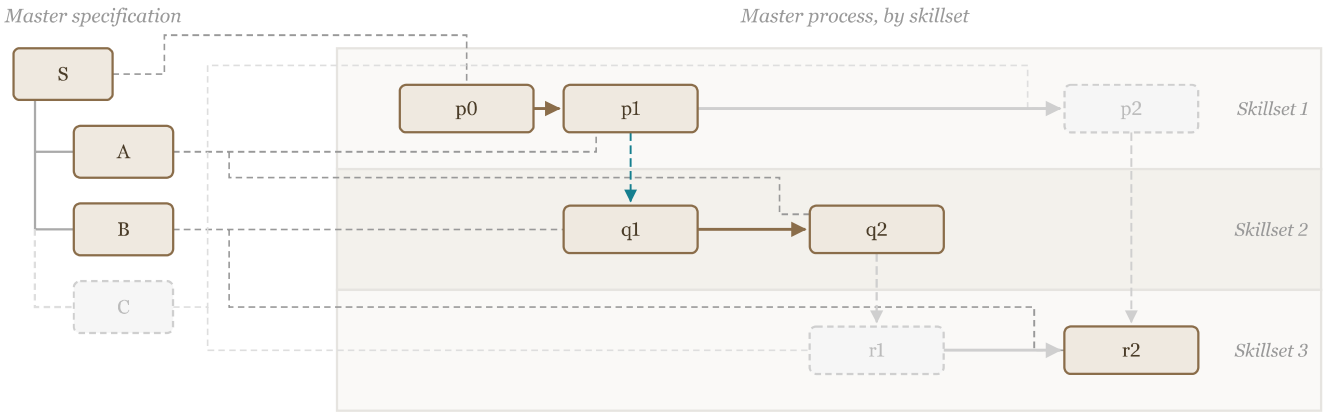


Figure 1. A master specification (left) and master process (right), the process laid out in skillset lanes. Grey dashed lines are the mapping from specification nodes to the steps they necessitate. Brown arrows carry work in progress between a skillset's steps, teal dashed arrows carry products between skillsets, and a step follows the one whose output it receives. A project's selected nodes and the steps they derive are drawn solid, and the rest of the master faded.

Take a single component of that selected subset, A, which necessitates p1 and q2 (Figure 2). A descriptor on A, x, scales the distribution of a metric y recorded at each of those steps (Figure 3). On a new project x is known from the specification but y is not, so the forecast is read from a fit to past records of the two (Figure 4).



Figure 2. Component A and the steps it necessitates, isolated from Figure 1 in the same positions and lanes, with the mapping drawn as grey dashed lines.

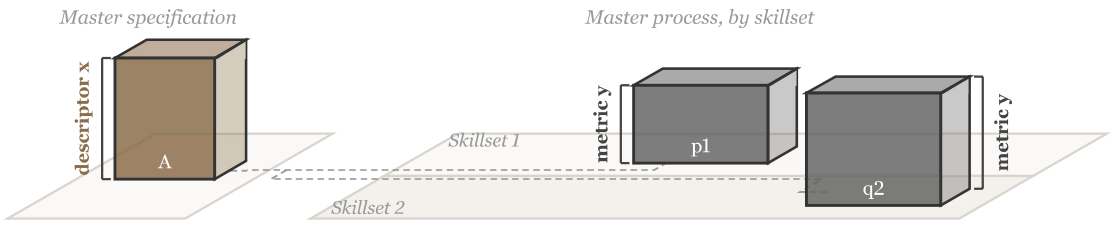


Figure 3. The same component and steps raised into three dimensions, each standing on its Figure 2 footprint. Height reads a value, descriptor x on A and metric y on each step, as the brackets mark.

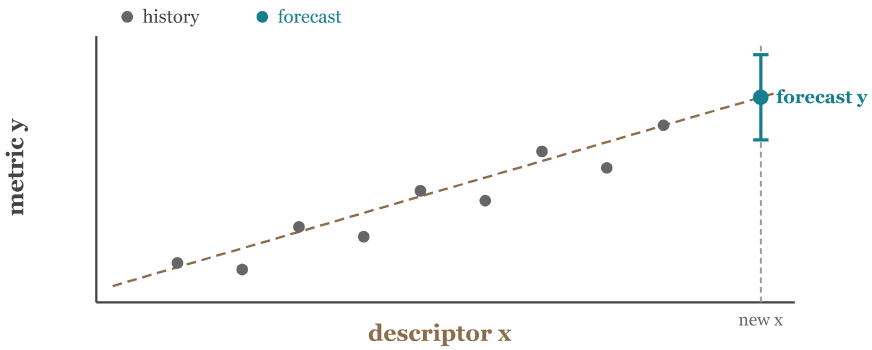


Figure 4. Grey points are past (descriptor x, metric y) records and the dashed line their fit. At a new project's descriptor x, the teal bar is the distribution the fit gives for metric y, and the forecast is read from it at the tolerated risk.

The figures reduce the forecast to a single descriptor, a single metric, and a straight-line fit, for clarity. In practice a step's outcome typically depends on several descriptors at once, and on the agents, tools and materials involved, so the fit is multivariate. Any method that maps recorded (descriptor, outcome) tuples to a predicted distribution serves, from multivariate regression to nearest-neighbour matching or a lightweight neural network, the established approaches Section 2 surveys. What the model contributes is the structured, attributed history those methods need.

3.3 Scope

The model sets aside the scheduling apparatus a reader from activity-on-node planning might expect. Resource capacity and levelling are not modelled, since allocation here is by self-selection rather than a planner assigning finite resources. The form of exposure, the scaling θ, the neighbourhood $\approx$ and the estimator are left open as modelling choices, and the experience floor λ_s, with the context over which experience is counted against it, is a policy input set per step that the model reads but does not define. Eligibility may also read external predicates for an agent's availability and documentation, which the model likewise takes as given. Forecasts are estimated from accumulated history, so the model gives no cold-start baseline, and a project with no relevant trace would need a descriptor-based one to seed them, as the worked examples note. Forecasts combine across a job's steps, and on to the project, under independence, which understates risk where converging paths share a source, a boundary the limitations take up.

4. Worked examples

We work two examples, a light-timber wall-frame panel and one in patient care. The second brings in three things the wall-frame example does not. The specification is extended during execution, the metric is an outcome relative to a dose of medication rather than time, and the setting is a second industry.

4.1 Wall-frame panel

The example is a light-timber wall-frame panel, optionally sheathed, delivered across the designer, machinist, and fabricator skillsets. The designer passes a cutting list to the machinist and a fabrication drawing to the fabricator, and the machinist passes framing members and sheathing panels to the fabricator. Figure 5 shows only the panel's selection P and its process G_P, not the masters in full. Numbers are illustrative, chosen to exercise the arithmetic, not measured.

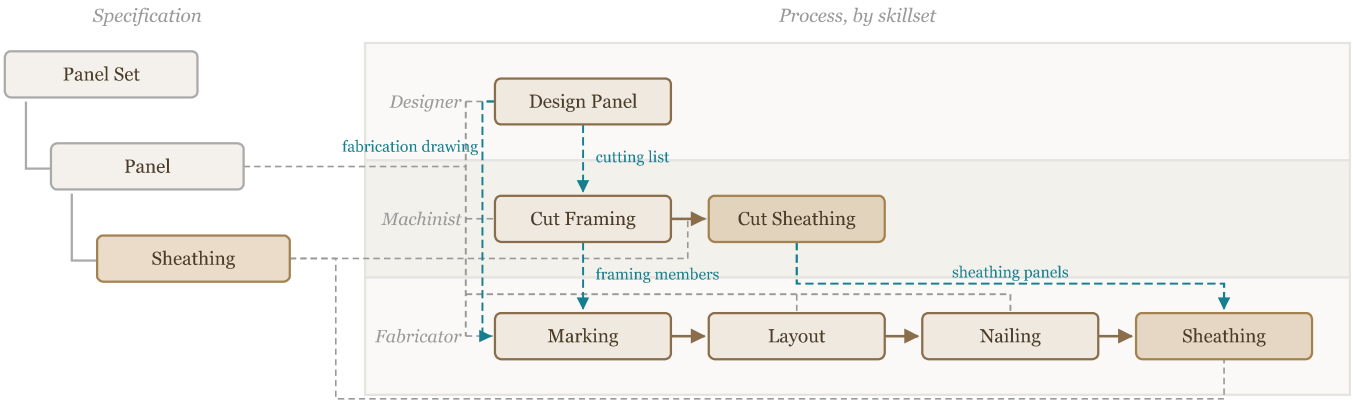
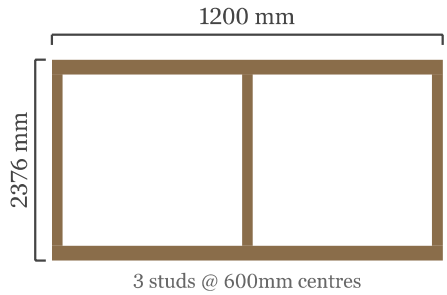


Figure 5. The panel's product tree and derived process, drawn as in Figure 1. The tinted elements are the Sheathing node and the steps it necessitates, present only when it is selected.

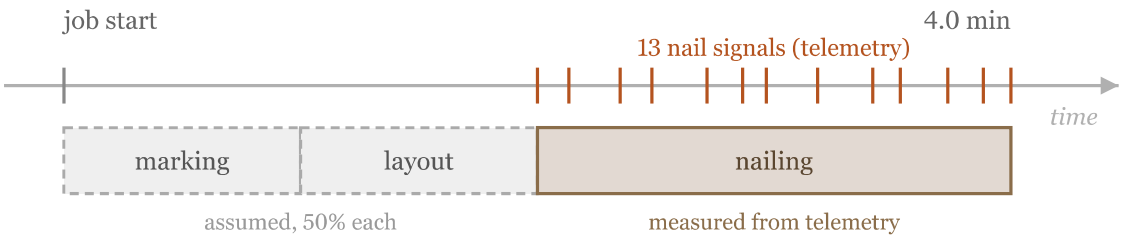
We now focus on the fabricator, who receives the machinist's framing members and the designer's fabrication drawing and assembles the members to that drawing.

Panel 1: the base frame



This specification resolves to 2 rails (horizontal) and 3 studs (vertical), 5 members, mapped to three fixed steps, marking, layout and nailing. Rails run the panel's width, and studs are cut to wall height.

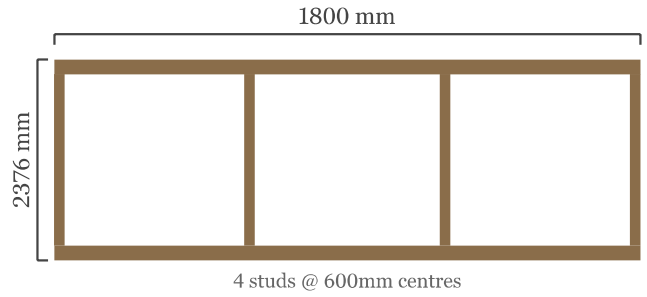
Quantity	Minimal (from spec)	Recorded
Wood	9528mm (2×1200 + 3×2376)	9710mm (+182mm waste, 1.9%)
Nails	12 (4 per stud)	13 (+1 misfire)
Time	—	4.0 min
Energy	—	0.55 kWh



Only the nailing step signals directly. Telemetry records all 13 nails fired at fairly regular intervals, which bounds that step's duration. Marking and layout emit no signal, so the time before the first nail is attributed to them, split evenly for want of anything finer.

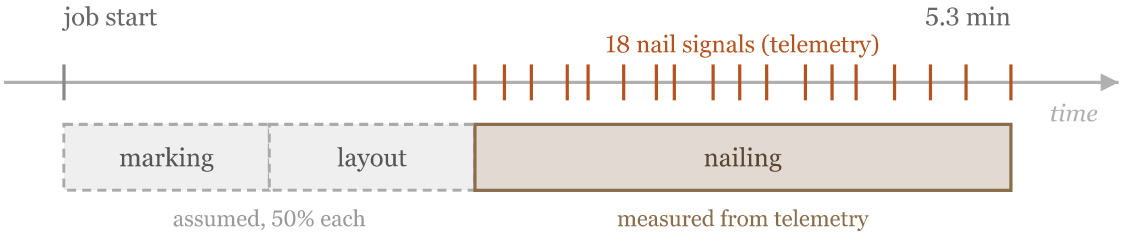
The minimal wood is fixed by the specification, and the recorded figure is what the panel consumed against it. The offcut making up the difference arises upstream at cutting rather than at assembly, so it is booked to the panel here rather than treated as the fabricator's own outcome.

Panel 2: a wider frame



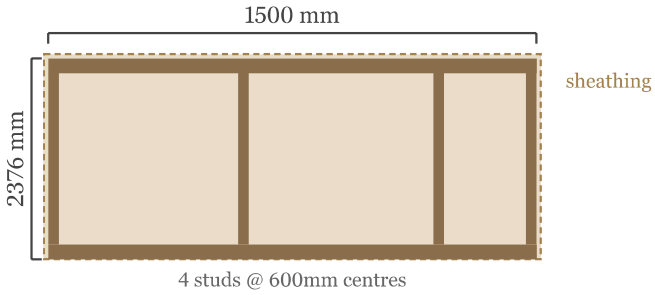
Panel width is the descriptor here. At 1800mm the specification resolves to 4 studs. Crossing a stud-spacing threshold adds a member, a cardinality-based extension rather than a value change on an existing component [4]. Width also scales the quantities, though not exactly, so it bears on the metric in both ways, in the member count and in what each member consumes.

Quantity	Minimal (from spec)	Recorded	Panel 1 scaled to this panel
Wood	13104mm (2×1800 + 4×2376)	13342mm (+238mm, 1.8%)	+250mm
Nails	16 (4 per stud)	18 (+2 misfires)	+1.3
Time	—	5.3 min	5.3 min
Energy	—	0.71 kWh	0.73 kWh



The last column scales Panel 1's figures to this panel, wood and nails by their minimums, and time and energy by stud count (×4/3), since framing effort tracks the members worked, as Panel 3 sets out. Recorded values sit close to these but not exactly on them, since panel width scales the metrics without fixing them, as Figure 3 shows.

Panel 3: forecasting the next frame



1500mm, 4 studs, sheathed. Sheathing is an optional feature, so the mapping adds its step only where the component is selected, unlike Panel 2's extra stud, a cardinality change to an existing step. All four steps are known from the specification with certainty. What the trace forecasts is the wastage on top of minimal quantities, and time and energy in full.

Wood and nail wastage are pooled across every prior frame. Time and energy condition on stud count rather than width, since framing effort tracks the number of members worked rather than their length, so only Panel 2, matching at 4 studs, is used.

Quantity	Minimal (from spec)	Forecast basis	Forecast
Wood	12504mm (2×1500 + 4×2376)	pooled waste rate across Panels 1 and 2 ≈ 1.86%	≈12780mm at $r = 0.9$ (centred ≈12737mm, +233mm)
Nails	16 (4 per stud)	pooled misfire rate across Panels 1 and 2: (1+2) / (12+16) ≈ 10.7%	≈18 (+2)
Time (framing)	—	Panel 2 only (matching stud count)	≈5.3 min
Energy (framing)	—	Panel 2 only (matching stud count)	≈0.71 kWh
Sheathing (time, energy)	—	no matching prior execution	no forecast yet

The pooled waste rate carries a spread, so wood has a distribution in the sense of Figure 4, centred near 12737mm. At a tolerated risk of $r = 0.9$ the forecast reads its 90th percentile, about 12780mm. The other rows show only the centre, for brevity.

Sheathing has no trace-based forecast. The specification places the step, but no recorded execution lies near this panel's descriptors, so the trace says nothing yet about what it costs here. A descriptor-based baseline could seed a forecast before any execution, but that is outside this example.

Panel 4: the contract offered to eligible workers

The fabricator's job is offered as one contract. Its inputs are the machinist's framing members and sheathing panels and the designer's fabrication drawing, its output is the panel, and its forecasts combine the framing steps' distributions. The wood offcut stays outside it, booked to the panel as Panel 1 set out. With the fabricator qualification assumed for this pool, eligibility turns on the experience floor at each step of the job.

Contract	Fabricator's job
Inputs	framing members, sheathing panels, fabrication drawing
Output	sheathed panel
Forecasts	framing ≈18 nails, ≈5.3 min, ≈0.71 kWh (centres), sheathing none yet

Step	Experience floor
Marking	≥5 recorded instances
Layout	≥5 recorded instances
Nailing	≥20 recorded framing instances
Sheathing	≥5 recorded sheathing instances

The floors are illustrative. Sheathing done on other product lines counts toward an agent's exposure at that step, so an agent can clear its floor here even though none of those records lies near enough to this panel to forecast it.

4.2 Patient care (illustrative)

In principle, the master specification covers the whole of clinical practice, rooted at the patient, whose descriptors, age, weight, ASA grade, attach to that root. Figures 6 to 8 show only a small fragment of it, around this case. A patient calls the clinic. The receptionist opens the case and, establishing that the complaint is abdomi-

nal, selects the abdominal region beneath the patient, with nothing further selected yet. That selection maps to an initial consult, in which the receptionist books the appointment and the GP assesses the patient (Figure 6).

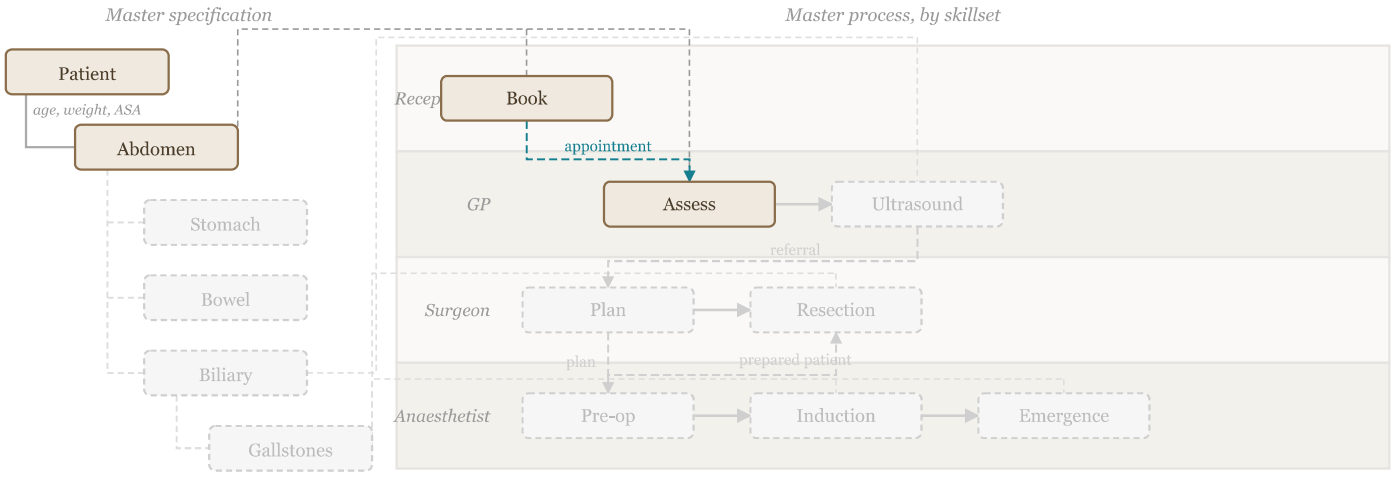


Figure 6. A fragment of the master specification and master process around this case, drawn as in Figure 1. The patient root carries the descriptors.

Examining the patient, the GP localises the pain to the biliary system and selects that organ, one level deeper. The deeper selection necessitates a further step, an ultrasound to image the organ, and the GP's consult extends to match (Figure 7). Nothing was added to the master. The biliary node was always there, and the assessment is what selected it.

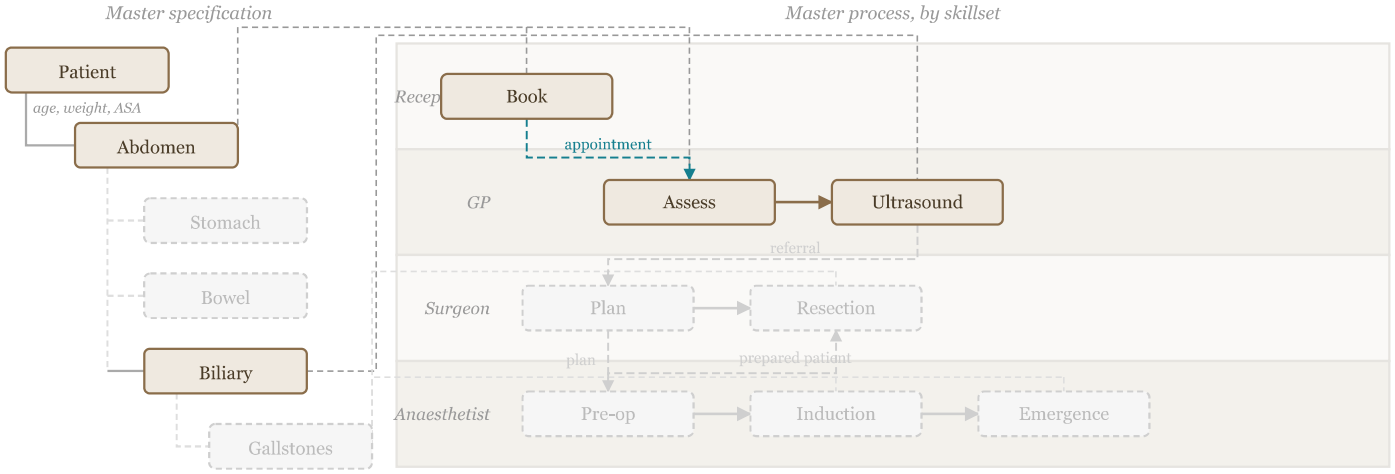


Figure 7. The GP's assessment selects the biliary node, which necessitates the ultrasound.

The ultrasound identifies gallstones, selecting the issue beneath the biliary organ. That selection reaches two skillsets the consult did not, the surgeon and the anaesthetist, and the process grows to match (Figure 8). This is the model's central move, and the same rule throughout. Each step's finding, its $\sigma(s)$, deepens the specification, and the deeper specification necessitates the next steps. The process progresses because the selection deepened, not because a plan was drawn. The order is inherited, so the booking, assessment and imaging already done stand.

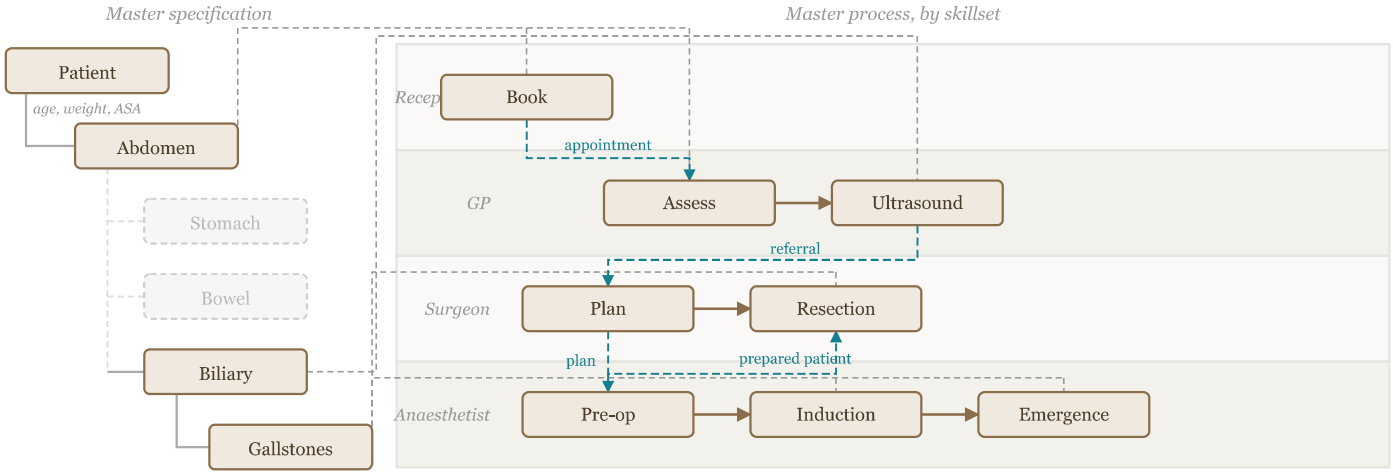


Figure 8. Imaging selects the gallstone node, which necessitates the surgeon's and anaesthetist's steps.

The induction brings in metrics the wall-frame example does not. It consumes a material, the anaesthetic drug, whose dose is recorded like any material quantity. Its outcome is measured not by time but by the largest fall in blood pressure after the drug is given. Eligibility gates the anaesthetist's job, and because cases vary widely, by weight, ASA grade and the like, exposure counted over comparable cases matters more here than raw case count.

Executing the step, the anaesthetist draws on that history to set the dose. For this patient's descriptors, the prior cases that match relate the dose given to the pressure drop that followed, showing which doses held blood pressure and which let it fall too far. The anaesthetist settles on a dose whose forecast drop, read at the tolerated risk, stays within a safe range (Figure 9). As in the wall frame, the distribution's spread is the risk being weighed, and the forecast is read from it at that risk. The fit is observational. The doses in the trace were chosen by clinicians for the patients in front of them, frailer patients receiving less, so the relation it shows is confounded by indication and cannot be read as the effect of a dose. Conditioning on age, weight and ASA grade narrows this without removing it, so the forecast informs the anaesthetist's choice rather than setting it.

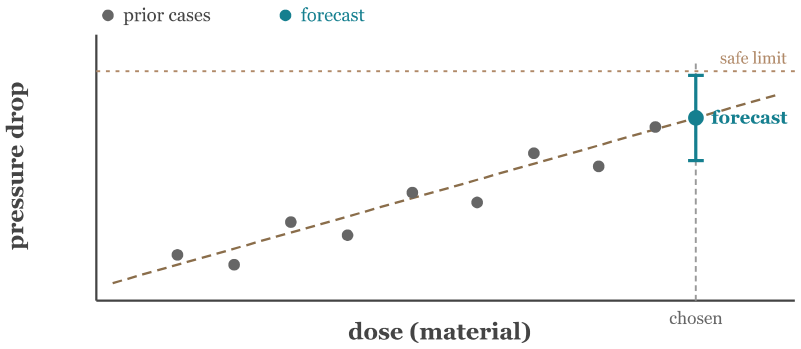


Figure 9. The anaesthetist's induction forecast, drawn as in Figure 4, with the dose given against the fall in blood pressure for prior cases with matching descriptors. The dotted line is the safe limit.

This is where the delivered process shows itself as a live set of instructions rather than a plan fixed in advance. It updates mid-execution, reactive to the conditions found on the ground. Opening the field, the surgeon finds a stone in the common bile duct that the ultrasound missed, selecting a duct-clearance node that was always in the master but not indicated until now. The surgeon's process gains an exploration step and the anaesthetist's maintenance extends to match, mid-procedure. The update is not only structural: the newly selected node binds a descriptor, a now-compromised liver, and reading it the anaesthetist selects cisatracurium, which

clears independently of the liver, in place of the default, a choice of material made at execution rather than fixed by the mapping. The order already set stands, the steps already completed hold, and nothing returns to the GP.

5. Evaluation

The model is implemented as a working platform, and the evaluation reported here is partial rather than conclusive. It verifies the model's mechanisms and shows that the model can be built over a real domain, but does not yet establish that derived processes are correct or that forecasts are accurate.

Mock data. Mock data was constructed from the author's experience of timber-frame manufacturing and construction, with master specifications, processes, agents and execution records modelled on real projects rather than generated at random. Runs against it exercise every mechanism of Section 3 end to end, from the derivation of a project's process from its selection, through extension of the specification during execution, capture of trace records including partial ones, forecasting at a chosen risk and the combination of forecasts into a contract's, and the exposure and experience floors that set eligibility, to preferential self-selection among eligible agents and generation of the contracts offered to them. In each case the platform behaves as Section 3 specifies. What the mock data cannot test is whether forecasts are accurate, since its records reflect the author's understanding of the work rather than independent observation of it.

Clinical data. We built the model over the MIMIC-IV Clinical Database Demo [36], an open subset of MIMIC-IV covering 100 patients and 275 admissions to a single academic medical centre. The master specification has 873 nodes over five levels, from the patient through body areas and structures to interventions, investigations and medications. The master process has 250 steps in 64 jobs, joined by 1,171 products passed between jobs, and the mapping links 854 of the 873 nodes to the steps they necessitate. All 275 admissions are represented as selections, with 15,497 dated events placed on the specification. Three admissions are replayed in four stages, and the derived process grows at each stage as the selection deepens. This shows that the formalism can be built over a real domain at this scale and that derivation runs end to end. It does not show that the derived processes are correct, and the specification, process and mapping were authored without clinical review, so their clinical validity is untested. The build also exposed gaps in the mapping. In 204 of the 275 admissions at least one contract receives no product, and in one replayed admission 829 nursing records fall on no job in the derived process.

The data is close to a natural trace. Its events are timestamped, many are attributed to individual caregivers or providers, drugs and doses are recorded as administered, and many fields are recorded for some events and not others. It supports four tests not yet run. Derivation can be tested by authoring the mapping on some admissions and measuring, on the rest, the share of recorded events the derived process accounts for and the share of derived steps that never occur. Forecasting can be tested by comparing forecasts formed at the level of the step with a baseline formed at the level of the whole admission, on held-out admissions, with calibration reported as the rate at which outcomes exceed the r-percentile against the nominal $1 - r$. Combined forecasts can be tested by comparing job-level forecasts made under independence with ones that preserve shared sources. The premise beneath eligibility, that recorded exposure to a step predicts performance at it, can be tested observationally. The demo's 100 patients are too few for the forecasting tests, which need the full database.

Two things the clinical data cannot test. Diagnoses are coded at discharge rather than when found, and the specification built here has no finding nodes, so selection driven by a step's findings, $\sigma(s)$, is not exercised, though timed results and reports could approximate it in a later build. Outcomes can be forecast and checked for calibration, but choosing treatments from those forecasts requires causal effects the observational record does not identify.

Court records. Ongoing testing aims to bring in court records, which by their nature hold detail anchored to a timeline, captured and published for the public record. Their master process is largely codified in rules of procedure, so derived processes can be checked against the docket, and cases deepen through motions and appeals much as patients do through findings. Where judges are assigned at random, they also allow an unconfounded test of whether recorded exposure predicts performance. Court data would barely exercise tools and materials, and its adversarial parties would stress-test the model's assumption that agents work toward the specification.

What the data in hand cannot test. Allocation is assigned rather than self-selected in both domains. Clinical staff are rostered, judges are assigned, and the lawyers' choice of cases goes unrecorded, so selection and contracts cannot yet be evaluated against real behaviour. The clinical data covers a single centre, and construction, the domain of the first worked example, awaits deployment with industrial partners.

6. Discussion and limitations

Per-step distributions combine into a job's forecasts, and on to the project, under the assumption that the steps are independent. That assumption holds well for a shallow graph with little sharing of sources. Where converging paths share a source, one agent or one material batch, their errors are correlated, and the independence assumption understates the risk, leaving the forecast at any tolerated risk optimistic. The effect grows with the number of paths converging through shared sources, which is to say with the size and interconnection of a project. Within the small-to-mid graphs SPD targets it is expected to be slight, and Monte Carlo sampling at the source level is the route past it, carrying a shared source through the paths it touches so their correlation is preserved. That route needs a trace attributed at the source level and is untested here. Independence can fail from the specification side too, where co-selected nodes interact, a roof with both a turret and a bay window being harder than either alone. The forecast captures this by conditioning on the combination where the trace holds such records, but those grow sparse as combinations multiply, and without them it falls back to an additive approximation that under-predicts interacting selections.

A second limitation is that derivation is additive. Extending the specification only adds nodes, and a selection that arrives after the steps it feeds have run, an extra socket after plastering, say, forces rework the model gives no account of. Nor can it express removal. Switching between alternatives in the specification, a slab foundation for blockwork, say, or a change order that replaces part of the product, removes nodes, and an additive model has no operation for it.

A third limitation is the granularity of the contract. A job is offered and taken whole, by one agent, so the model does not split a job between executors, one tasked with one of its outputs and another with a separate one, nor run instances of the same step in parallel. Both are common in practice, and both raise questions of order and attribution the model leaves for later work.

7. Conclusion and future work

TODO.

References

[1] van der Aalst, W. M. P. (1999). On the automatic generation of workflow processes based on product structures. *Computers in Industry*, *39*(2), 97-111.

[2] Reijers, H. A., Limam, S., & van der Aalst, W. M. P. (2003). Product-based workflow design. *Journal of Management Information Systems*, *20*(1), 229-262.

[3] Kang, K. C., Cohen, S. G., Hess, J. A., Novak, W. E., & Peterson, A. S. (1990). *Feature-oriented domain analysis (FODA) feasibility study* (Technical Report CMU/SEI-90-TR-021). Software Engineering Institute, Carnegie Mellon University.

[4] Czarnecki, K., & Kim, C. H. P. (2005). Cardinality-based feature modeling and constraints: A progress report. In *International Workshop on Software Factories (OOPSLA)* (pp. 16-20).

[5] Pohl, K., Böckle, G., & van der Linden, F. (2005). *Software product line engineering: Foundations, principles, and techniques*. Springer. https://doi.org/10.1007/3-540-28901-1

[6] van der Aalst, W. M. P., Weske, M., & Grünbauer, D. (2005). Case handling: A new paradigm for business process support. *Data & Knowledge Engineering*, *53*(2), 129-162.

[7] Vanderfeesten, I., Reijers, H. A., & van der Aalst, W. M. P. (2007). An evaluation of case handling systems for product based workflow design. In *Proceedings of ICEIS 2007* (Vol. 3, pp. 39-46).

[8] Kelley, J. E., & Walker, M. R. (1959). Critical-path planning and scheduling. In *Proceedings of the Eastern Joint Computer Conference* (pp. 160-173).

[9] Pantelides, C. C. (1994). Unified frameworks for optimal process planning and scheduling. In *Foundations of Computer-Aided Process Operations* (pp. 253-274). CACHE Publications.

[10] Kondili, E., Pantelides, C. C., & Sargent, R. W. H. (1993). A general algorithm for short-term scheduling of batch operations - I. MILP formulation. *Computers & Chemical Engineering*, *17*(2), 211-227.

[11] Omicini, A., Ricci, A., & Viroli, M. (2008). Artifacts in the A&A meta-model for multi-agent systems. *Autonomous Agents and Multi-Agent Systems*, *17*(3), 432-456.

[12] International Organization for Standardization. (2018). *Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries — Part 1: Data schema (ISO 16739-1:2018)*. International Organization for Standardization.

[13] Márquez-Chamorro, A. E., Resinas, M., & Ruiz-Cortés, A. (2018). Predictive monitoring of business processes: A survey. *IEEE Transactions on Services Computing*, *11*(6), 962-977.

[14] Vanhoucke, M. (2009). *Measuring time: Improving project performance using earned value management*. Springer.

[15] Vanhoucke, M. (2012). *Project management with dynamic scheduling: Baseline scheduling, risk analysis and project control*. Springer.

[16] Vanhoucke, M. (2015). On the use of schedule risk analysis for project management. *The Journal of Modern Project Management*, *2*(3), 108-117.

[17] Malcolm, D. G., Roseboom, J. H., Clark, C. E., & Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research*, *7*(5), 646-669.

[18] Wauters, M., & Vanhoucke, M. (2016). A comparative study of artificial intelligence methods for project duration forecasting. *Expert Systems with Applications*, *46*, 249-261.

[19] Wauters, M., & Vanhoucke, M. (2017). A nearest neighbour extension to project duration forecasting with artificial intelligence. *European Journal of Operational Research*, *259*(3), 1097-1111.

[20] Flyvbjerg, B. (2006). From Nobel Prize to project management: Getting risks right. *Project Management Journal*, *37*(3), 5-15.

[21] Batselier, J., & Vanhoucke, M. (2016). Practical application and empirical evaluation of reference class forecasting for project management. *Project Management Journal*, *47*(5), 36-51.

[22] Batselier, J., & Vanhoucke, M. (2017). Improving project forecast accuracy by integrating earned value management with exponential smoothing and reference class forecasting. *International Journal of Project Management*, *35*(1), 28-43.

[23] Batselier, J., & Vanhoucke, M. (2015). Construction and evaluation framework for a real-life project database. *International Journal of Project Management*, *33*(3), 697-710.

[24] Vanhoucke, M. (2023). *The illusion of control: Project data, computer algorithms and human intuition for project management and control*. Springer.

[25] Der Kiureghian, A., & Ditlevsen, O. (2009). Aleatory or epistemic? Does it matter? *Structural Safety*, *31*(2), 105-112.

[26] Malone, T. W., & Crowston, K. (1994). The interdisciplinary study of coordination. *ACM Computing Surveys*, *26*(1), 87-119.

[27] Grassé, P.-P. (1959). La reconstruction du nid et les coordinations interindividuelles chez Bellicositermes natalensis et Cubitermes sp. La théorie de la stigmergie: Essai d'interprétation du comportement des termites constructeurs. *Insectes Sociaux*, *6*(1), 41-80.

[28] Heylighen, F. (2016). Stigmergy as a universal coordination mechanism I: Definition and components. *Cognitive Systems Research*, *38*, 4-13.

[29] Heylighen, F. (2016). Stigmergy as a universal coordination mechanism II: Varieties and evolution. *Cognitive Systems Research*, *38*, 50-59.

[30] Christensen, L. R. (2013). Stigmergy in human practice: Coordination in construction work. *Cognitive Systems Research*, *21*, 40-51.

[31] Masood, Z., Hoda, R., & Blincoe, K. (2017). Motivation for self-assignment: Factors agile software developers consider. In *2017 IEEE/ACM 10th International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)* (pp. 92-93).

[32] Masood, Z., Hoda, R., & Blincoe, K. (2021). What drives and sustains self-assignment in agile teams. *IEEE Transactions on Software Engineering*, *48*(9), 3626-3639.

[33] Conoscenti, M., Besner, V., Vetrò, A., & Méndez Fernández, D. (2019). Combining data analytics and developers feedback for identifying reasons of inaccurate estimations in agile software development. *Journal of Systems and Software*, *156*, 126-135.

[34] Jørgensen, M., & Gruschke, T. M. (2009). The impact of lessons-learned sessions on effort estimation and uncertainty assessments. *IEEE Transactions on Software Engineering*, *35*(3), 368-383.

[35] Anvik, J., Hiew, L., & Murphy, G. C. (2006). Who should fix this bug? In *Proceedings of the 28th International Conference on Software Engineering (ICSE 2006)* (pp. 329-338).

[36] Johnson, A., Bulgarelli, L., Pollard, T., Horng, S., Celi, L. A., & Mark, R. (2023). MIMIC-IV Clinical Database Demo (version 2.2). PhysioNet. https://doi.org/10.13026/dp1f-ex47